EC-220 Computer System Architecture

Dr. Ahsan Shahzad

Department of Computer and Software Engineering (DC&SE)

College of Electrical and Mechanical Engineering (CEME)

National University of Science and Technology (NUST)

Week # 5_Corona
Break

MIPS ISA

- **Microprocessor without Interlocked Pipeline Stages (MIPS)**
- Thirty-two **32-bit** general-purpose registers (GPR).
- **Register $0** is hardwired to zero and writes to it are discarded.
- **Register $31** is the link register.
- The **program counter (PC)** specifies the address of the next Instruction.
- The **exception program counter (EPC)** register remembers the program counter when there is an interrupt or exception.
- For integer multiplication and division instructions, which run asynchronously from other instructions, a pair of 32-bit registers, **HI and LO** are provided.

MIPS Instruction Format

- Instructions are divided into three types: **R, I and J.**
- Every instruction starts with a **6-bit opcode.**

Type	-31- format (bits) -0-					
R	opcode (6)	rs (5)	rt (5)	rd (5)	shamt (5)	funct (6)
I	opcode (6)	rs (5)	rt (5)	immediate (16)		
J	opcode (6)	address (26)				

In addition to the opcode,

- **R-type** instructions specify **three registers, a shift amount field, and a function field;**
- **I-type** instructions specify **two registers** and a **16-bit immediate value**;
- **J-type** instructions follow the opcode with a **26-bit jump target**

MIPS Assembly and Instructions Format

- **R-format**
  - **OP rd, rs, rt** **OR** **OP rd, rt, shamt**
  - EX: **add $S1, $S2, $S3** OR **sll $S1, $S2, 2**

- **I-format**
  - **OP rt, IMM(rs)** **OR** **OP rs, rt, IMM**
  - EX: **lw $S1, 4($S2)** OR **addi $S1, $S2, 6**

- **J-format**
  - **OP Label**
  - EX: **J Top_Label**

Type	-31- format (bits) -0-					
R	opcode (6)	rs (5)	rt (5)	rd (5)	shamt (5)	funct (6)
I	opcode (6)	rs (5)	rt (5)	immediate (16)		
J	opcode (6)	address (26)				

32-Bit Constants and lui

Type	-31-	format (bits)				-0-
R	opcode (6)	rs (5)	rt (5)	rd (5)	shamt (5)	funct (6)
I	opcode (6)	rs (5)	rt (5)	immediate (16)		
J	opcode (6)	address (26)				

- It is easy to load a register with a constant from **-32768 to 32767**, e.g.,

ori $t0, $0, 42

- Larger numbers use **"load upper immediate (lui)",** which fills a register with a 16-bit immediate value followed by 16 zeros; an OR inst. handily fills in the rest. E.g., load $t0 with **0xCODEFACE:**

lui $t0, 0xCODE

ori $t0, $t0, 0xFACE

- The assembler automatically expands **the li pseudo-instr**. Into such an instr. sequence

li $t1, 0xCAFE0B0E → lui $t1, 0xCAFE

ori $t1, $t1, 0x0B0E

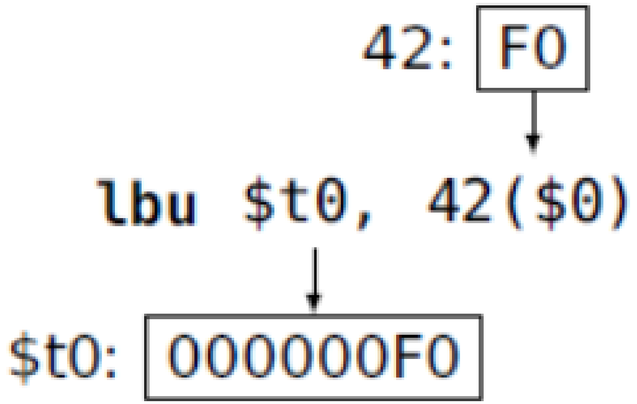
Load and Store Instructions

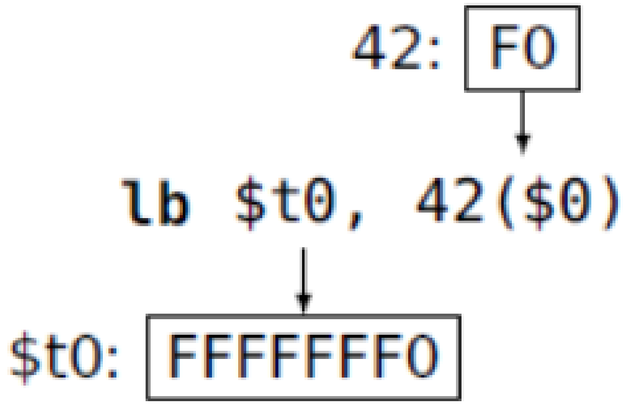
Load/Store Instructions	
`lb`	Load byte
`lbu`	Load byte unsigned
`lh`	Load halfword
`lhu`	Load halfword unsigned
`lw`	Load word
`lwl`	Load word left
`lwr`	Load word right
`sb`	Store byte
`sh`	Store halfword
`sw`	Store word
`swl`	Store word left
`swr`	Store word right

- The MIPS is a load/store architecture: data must be moved into registers for computation.
- Other architectures e.g. x86 allow arithmetic directly on data in memory.

Byte Load and Store

- MIPS registers are 32 bits (4 bytes).
- Loading a byte into a register either clears the top three bytes or sign-extends them.



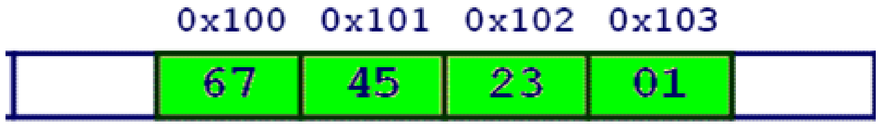


The Endian Question

Big Endian

MIPS can also load and store 4-byte words and 2-byte halfwords.

The *endian* question:



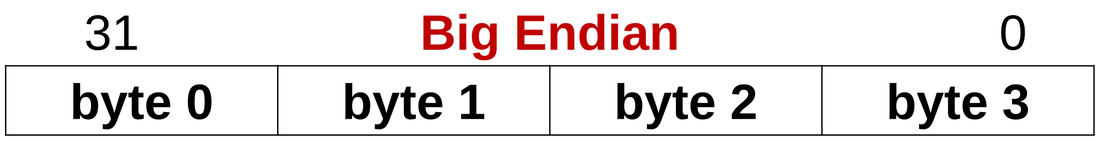
when you read a word, in what order do the bytes appear?

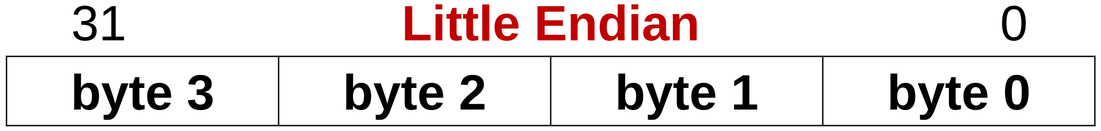
- **Little Endian:** Intel, DEC, et al.
- **Big Endian:** Motorola, IBM, Sun, et al.

Little Endian

01234567

- MIPS can do either; SPIM adopts its host's convention
- The newer versions of most processors shifts to Bi-endian format e.g. Powerpc (Motorola), Alpha (DEC), Spark (Sun), ARM etc.





Testing Endianness

```
            .data                   # Directive: “this is data”
myword:     .word 0                 # Define a word of data (=0)
            .text                   # Directive: “this is program”
main:
            la $t1, myword          # pseudoinstruction: load address
            li $t0, 0x11
            sb $t0, 0($t1)          # Store 0x11 at byte 0
            li $t0, 0x22
            sb $t0, 1($t1)          # Store 0x22 at byte 1
            li $t0, 0x33
            sb $t0, 2($t1)          # Store 0x33 at byte 2
            li $t0, 0x44
            sb $t0, 3($t1)          # Store 0x44 at byte 3
            lw $t2, 0($t1)          # 0x11223344 or 0x44332211?
            j $ra
```

Alignment

Word and half-word loads and stores must be aligned:

- Words must start at a multiple of 4 bytes
- Half-words on a multiple of 2
- Byte load / store has no such constraint.

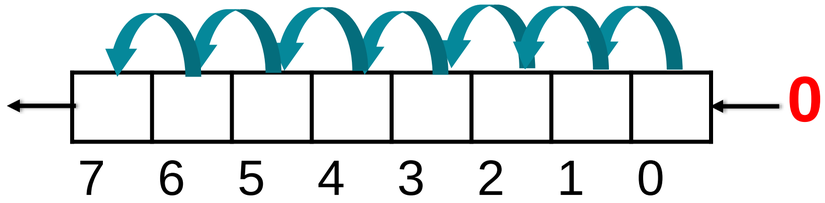
```
lw $t0,  4($0) # OK
lw $t0,  5($0) # BAD: 5 mod 4 = 1
lw $t0,  8($0) # OK
lw $t0, 12($0) # OK

lh $t0,  2($0) # OK
lh $t0,  3($0) # BAD: 3 mod 2 = 1
lh $t0,  4($0) # OK
```

Logical Instructions

Logical operations	C operators	Java operators	MIPS instructions
Shift left	<<	<<	sll
Shift right	>>	>>>	srl
Bit-by-bit AND	&	&	and, andi
Bit-by-bit OR	|	|	or, ori
Bit-by-bit NOT	~	~	nor

- **AND** -> clear some bits whereas, **OR** -> to set some bits
- MIPS also provides the instructions *and immediate* (andi) and *or immediate* (ori).
- **NOR: A NOR 0** = NOT (A OR 0) = **NOT (A).**

Shift Left - Logical Instructions



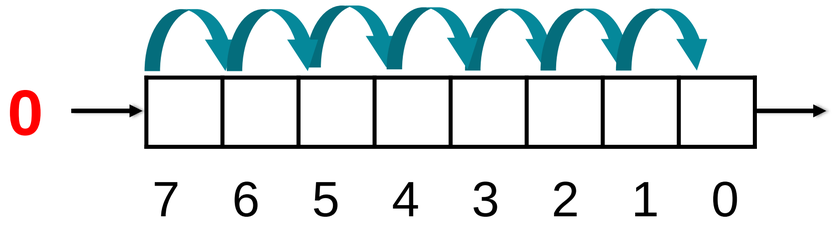
- **sll rd, rt, 4** # reg $d = reg $t << 4 bits
  - E.g. sll $t2, $s0, 4 # reg $t2 = reg $s0 << 4 bits

Op	rs	rt	rd	shamt	func
0	**0**	16	10	4	**0**

- **sllv rd, rt, rs** # **Shift left logical variable**; no. of bits to be shifted is provided via register.
  - E.g. sllv $t2, $s0, $t0

Op	rs	rt	rd	shamt	func
0	**8**	16	10	**0**	**4**

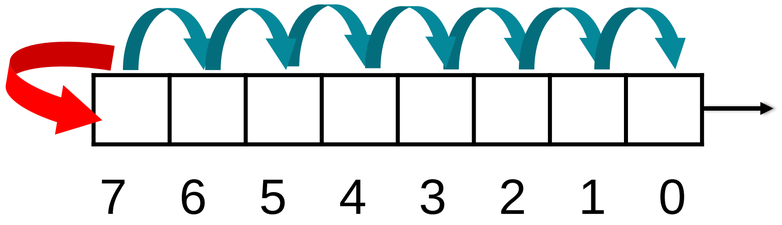
Shift Right - Logical Instructions



- **Shift Right Logical**
  - srl rd, rt, 1 # reg $d = reg $t >> 1 bit
  - E.g. srl $t2, $s0, 1 # reg $t2 = reg $s0 >> 1 bit

Op	rs	rt	rd	shamt	func
0	**0**	16	10	1	**2**

- **Shift Right Arithmetic**
  - sra rd, rt, 3 # re-insert the msb or sign extends
  - E.g. sra $t2, $s0, 3



Decision-Making Instructions OR Conditional Jumps

- MIPS assembly language includes two decision-making instructions, traditionally called **conditional branches**.

beq register1, register2, L1

bne register1, register2, L1

- **Beq:** if(rs == rt) pc+= 16 bit signed offset * 4 (**PC-relative addressing**)
- **No. of instructions to be skipped** mentioned here as **offset** and not the actual bytes count. O**ffset x 4** and then **add with PC**

Decision Making Instructions

- **For greater or less comparisons,**

```
slt $t0, $s3, $s4     # $t0 = 1 if $s3 < $s4

slti $t0,$s2,10       # $t0 = 1 if $s2 < 10
```

- MIPS compilers use the **slt, slti, beq, bne,** and the fixed value of 0 (always available by reading register **$zero) to create all relative conditions**: equal, not equal, less than, less than or equal
- There **isn't a status register (Flag Register).** Instead, the results of a comparison set a register value. The branch then tests this register value.

Other Decision Making Pseudo-Instructions

- **Another family of branches tests a single register:**

```
            bgez  $t0,  myelse    # Branch if $t0 positive
             # . . .
        myelse:
            # . . .
```

- Others in this family:

- **blez** Branch on less than or equal to zero
- **bgtz** Branch on greater than zero
- **bltz** Branch on less than zero
- **bltzal** Branch on less than zero and link
- **bgez** Branch on greater than or equal to zero
- **bgezal** Branch on greater than or equal to zero and link

➢ "Link" variants will always save the address of next instruction into $ra, just like **jal.**

Assembler Pseudo-instructions

Branch always	**b** *label*	→	**beq** $0, $0, *label*
Branch if equal zero	**beqz** s, *label*	→	**beq** s, $0, *label*
Branch greater or equal	**bge** s, t, *label*	→	**slt** $1, s, t **beq** $1, $0, *label*
Branch greater or equal unsigned	**bgeu** s, t, *label*	→	**sltu** $1, s, t **beq** $1, $0, *label*
Branch greater than	**bgt** s, t, *label*	→	**slt** $1, t, s **bne** $1, $0, *label*
Branch greater than unsigned	**bgtu** s, t, *label*	→	**sltu** $1, t, s **bne** $1, $0, *label*
Branch less than	**blt** s, t, *label*	→	**slt** $1, s, t **bne** $1, $0, *label*
Branch less than unsigned	**bltu** s, t, *label*	→	**sltu** $1, s, t **bne** $1, $0, *label*

Assembler Pseudo-instructions

Load immediate $0 \le j \le 65535$	**li** *d*, *j*	→	**ori** *d*, $0, *j*
Load immediate $-32768 \le j < 0$	**li** *d*, *j*	→	**addiu** *d*, $0, *j*
Load Immediate	**li** *d*, *j*	→	**lui** *d*, hi16 (*j*) **ori** *d*, *d*, lo16 (*j*)
Move	**move** *d*, *s*	→	**or** *d*, *s*, $0
Multiply	**mul** *d*, *s*, *t*	→	**mult s, t** **mflo d**
Negate unsigned	**negu** *d*, *s*	→	**subu** *d*, $0, *s*
Set if Equal	**seq** *d*, *s*, *t*	→	**xor** *d*, *s*, *t* **sltiu** *d*, *d*, 1
Set if greater or equal	**sge** *d*, *s*, *t*	→	**slt** *d*, *s*, *t* **xori** *d*, *d*, 1
Set if greater or equal unsigned	**sgeu** *d*, *s*, *t*	→	**sltu** *d*, *s*, *t* **xori** *d*, *d*, 1
Set if greater than	**sgt** *d*, *s*, *t*	→	**slt** *d*, *t*, *s*

Subroutines

- a.k.a procedures, functions, methods, et al.
- Code that can run by itself, then resume whatever invoked it.
- Exist for three reasons:
  - **Code reuse:**
    - Function libraries
    - Recurring computations aside from loops
  - **Abstraction/Isolation:**
    - What happens in a function stays in the function.
  - **Enabling Recursion:**
    - Fundamental to divide-and-conquer algorithms.

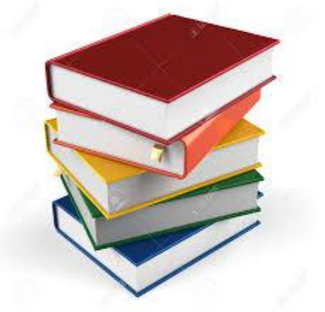
Supporting Procedures In Computer Hardware

- MIPS used following conventions for Procedure Calling,
  - **$a0 - $a3:** four argument registers
  - **$v0-$v1:** two return value registers
  - **$ra:** one return address register
- **Jal ProcedureAddress:** is used to jump to procedure and simultaneously saving the address of next instruction.
- **Jr $ra:** Jump register instruction is used to return control back to the caller

MIPS Register Usage Conventions

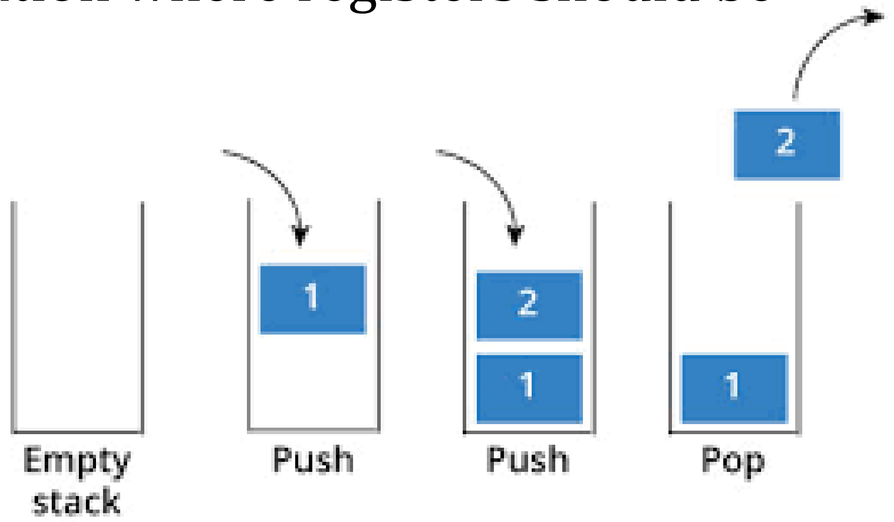
Register	Name	Function	Reserved ?
R0	**$zero**	Always contains constant 0	N.A.
R1	**$at**	**Reserved for Assembler temporary**	N.A.
R2-R3	**$v0-$v1**	Expression evaluation and procedure return values	No
R4-R7	**$a0-$a3**	Function arguments	No
R8-R15	**$t0-$t7**	For temporary values	**No**
R16-R23	**$s0-$s7**	Saved registers across function calls	**Yes**
R24-R25	**$t8-$t9**	More temporary values	No
R26-R27	**$k0-$k1**	**Reserved for interrupt handler / OS**	N.A
R28	**$gp**	Global pointer	Yes
R29	**$sp**	Stack Pointer	Yes
R30	**$fp**	Frame pointer	Yes
R31	**$ra**	Return address from function call	Yes

Stacks



➢ **If compiler needs more registers for a Procedure, then what?**

- **Stack:** A data structure for spilling registers organized as a last-in-first-out queue.
- **Stack Pointer ($SP, r29):** holds the address of stack location where registers should be spilled or where old register values can be found.
- **PUSH:** Add element to stack.
- **POP:** Remove element from the stack.



- Historically, stacks “grow” from higher addresses to lower addresses.

Procedures Example

Compiling a **Leaf_procedure** (that doesn't call other procedures)

```
int  leaf_example (int g, int h, int i, int j)
{
int f ;
f = (g + h) - (i + j) ;
return f ;
}
```

The parameter variables **g, h, i, and j** correspond to the argument registers **$a0, $a1, $a2, and $a3,** and **f corresponds to $s0**. The compiled program starts with the label of the procedure:

Procedures Example

leaf_example:

```
addi $sp, $sp, –12    # adjust stack to make room for 3 items
sw $t1, 8($sp)        # save register $t1 for use afterwards
sw $t0, 4($sp)        # save register $t0 for use afterwards
sw $s0, 0($sp)        # save register $s0 for use afterwards

add $t0,$a0,$a1       # register $t0 contains g + h
add $t1,$a2,$a3       # register $t1 contains i + j
sub $s0, $t0, $t1     # f = $t0 – $t1, which is (g + h)–(i + j)
```

Procedures Example

```
# returns f ($v0 = $s0 + 0)   MOVE instr.
    add $v0, $s0, $zero

# restore registers $s0, $t0, $t1 for caller
    lw $s0, 0($sp)
    lw $t0, 4($sp)
    lw $t1, 8($sp)
# adjust stack to delete 3 items
    addi $sp,$sp,12

# jump back to calling routine
    jr $ra
```

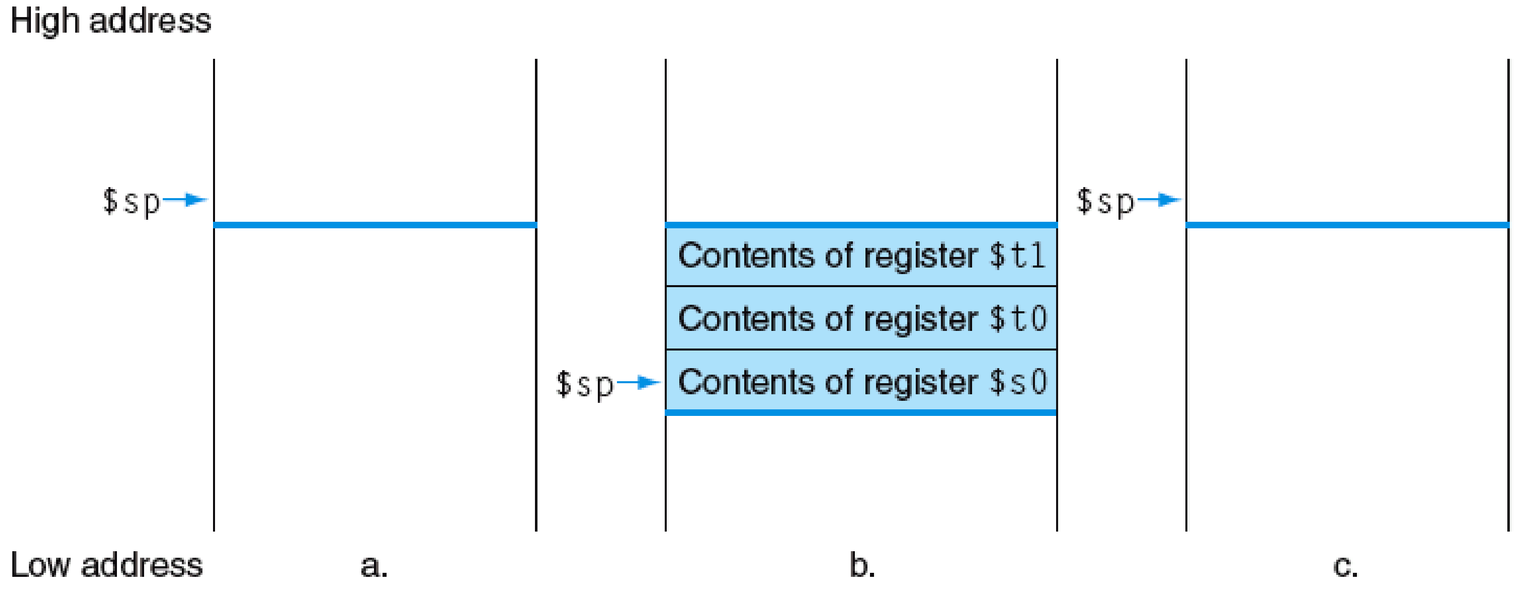


FIGURE 2.14 The values of the stack pointer and the stack (a) before, (b) during, and (c) after the procedure call. The stack pointer always points to the "top" of the stack, or the last word in the stack in this drawing.

Procedures Example

Compiling a **Nested_procedure** (Non-leaf procedure)

```
int fact (int n)
{
if (n < 1)
        return (1);
else
        return (n* fact(n - 1));
}
```

The parameter variables **n** correspond to the argument registers **$a0.**

What is the MIPS assembly code?

Procedures Example

```
fact:
    addi $sp, $sp, –8    # adjust stack to make room for 2 items
    sw $ra, 4($sp)       # save the return address
    sw $a0, 0($sp)       # save the argument n

    Slti $t0,$a0,1       # test for n < 1?
    beq $t0,$zero, L1    # if n >=1, go to L1
    addi $v0, $zero, 1   # yes, return 1
    Addi $sp, $sp, 8     # pop 2 item off stack
    Jr $ra               # return to caller
L1:
    addi $a0, $a0, –1    # n > = 1: argument gets (n-1)
    Jal fact             # Recursive call fact with (n-1)
```

Procedures Example

```
# the next instr. Is where fact returns
lw $a0, 0($sp)          # return from jal: restore argument n
lw $ra, 4($sp)          # restore the return address
addi $sp, $sp, 8        #adjust stack pointer to pop 2 items

mul $v0, $a0, $v0       # compute n * fact(n-1)

Jr  $ ra  # return to the caller
```

Differences in other ISAs

- More or fewer general-purpose registers (E.g., Itanium: 128, 6502: 3)
- Arithmetic instructions affect condition codes (e.g., carry, zero); conditional branches test these flags
- More addressing modes (e.g., x86: 6; VAX: 20)
- Registers that are more specialized (e.g., x86)
- Arithmetic instructions that also access memory (e.g., x86, VAX)
- Arithmetic instructions on other data types (e.g. bytes and halfwords)
- Variable-length instructions (e.g. x86; ARM)
- Predicated Instructions (e.g. ARM, VLIW)
- Single instructions that do much more (e.g., x86 string move, procedure entry/exit)

Character Representations

ASCII Codes (American Standard Code for Information Interchange)

- Most popular encoding scheme
- Only 7 bits are used for ASCII codes -> 128 ASCII Codes
- 0-31 and 127 are for communication control purposes
- 32-126 -> 95 ASCII codes are printable
- E.g. A = 41h , Z = 59h, a = 61h, Space = 20h etc.

ASCII Codes

	0	1	2	3	4	5	6	7
0:	NUL '\0'	DLE		0	@	P	`	p
1:	SOH	DC1	!	1	A	Q	a	q
2:	STX	DC2	"	2	B	R	b	r
3:	ETX	DC3	#	3	C	S	c	s
4:	EOT	DC4	$	4	D	T	d	t
5:	ENQ	NAK	%	5	E	U	e	u
6:	ACK	SYN	&	6	F	V	f	v
7:	BEL '\a'	ETB	'	7	G	W	g	w
8:	BS '\b'	CAN	(	8	H	X	h	x
9:	HT '\t'	EM	)	9	I	Y	i	y
A:	LF '\n'	SUB	*	:	J	Z	j	z
B:	VT '\v'	ESC	+	;	K	[	k	{
C:	FF '\f'	FS	,	<	L	\	l	|
D:	CR '\r'	GS	-	=	M	]	m	}
E:	SO	RS	.	>	N	^	n	~
F:	SI	US	/	?	O	_	o	DEL

Home Work Tasks

We will discuss these points during our interactive session via Microsoft Teams

- Does Endianness affects file formats?
- Find out the Endianness of your PC
- Please go through remaining MIPS Instructions in your free time from online resources.